

Report - Version 5: Simple Deep Neural Network

Dataset	Task	Architecture	Test acc.
Fashion-MNIST	10 classes	784 -> 128 -> 64 -> 10	89.92%

Introduction

Version 5 extends the mini-batch network from Version 4 by adding a second hidden layer. The training strategy remains the same, while the model becomes deeper and can transform the input through two successive ReLU representations before the final softmax output.

Goal of Version 5

The main goal is to understand how a deeper feed-forward network is built from the same simple components already introduced in the project: matrix multiplication, biases, ReLU, softmax, cross-entropy, mini-batches, and backpropagation.

Data and Preparation

The project uses Fashion-MNIST with 60,000 labeled training images and 10,000 labeled test images. Each 28 x 28 grayscale image is flattened into 784 pixel values. Pixel values are normalized from 0-255 to 0-1 and labels are converted to one-hot vectors for the 10 classes.

Architecture

Version 5 adds a second hidden layer. The first hidden layer has 128 neurons and the second has 64 neurons. Both use ReLU. The output layer has 10 neurons followed by softmax.

784 pixels -> 128 hidden neurons -> ReLU -> 64 hidden neurons -> ReLU -> 10 output scores -> softmax -> argmax

Deeper Forward Pass

Each mini-batch first passes through the 128-neuron hidden layer, then through the 64-neuron hidden layer, and finally through the 10-neuron output layer. ReLU is applied after each hidden layer and softmax converts the final scores into class probabilities.

Mini-Batch Training

Training keeps the Version 4 strategy: the data is shuffled at the beginning of every epoch, split into mini-batches of 64 examples, and the parameters are updated after every mini-batch.

Backpropagation Through Three Layers

The output error is first used to compute gradients for w_3 and b_3 . The error is then propagated backward through the second ReLU layer to compute gradients for w_2 and b_2 , and finally through the first ReLU layer to compute gradients for w_1 and b_1 .

Loss and Numerical Safety

Multiclass cross-entropy measures the prediction error. Predicted probabilities are clipped to the range from $1e-12$ to 1.0 before applying the logarithm, preventing $\log(0)$ and keeping the loss computation numerically safe.

Training

Training uses learning rate 0.1 , 200 fixed epochs, and batch size 64. The reported training loss is the mean of the mini-batch losses from the final epoch.

Training Result

After 200 epochs, the mean loss of the final epoch is 0.0145 and training accuracy is 99.59% .

Evaluation

The full training set and test set are evaluated using the complete $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ forward pass. The predicted class is selected with argmax from the softmax probabilities. Test accuracy is 89.92% .

Results

Metric	Value
Final epoch mean loss	0.0145
Training accuracy	99.59%
Test accuracy	89.92%

What Version 5 Teaches

Version 5 shows how the same learning process can be extended to a deeper network. Adding a hidden layer requires one additional forward transformation and one additional backward gradient step, while the overall training logic remains unchanged.

Current Performance

With this configuration, the network reaches 99.59% training accuracy and 89.92% test accuracy on Fashion-MNIST.

Generalization Gap

Training accuracy is higher than test accuracy. This difference shows that the network fits the training data more closely than the unseen test data.

Limitations

The model still uses a fixed learning rate, plain mini-batch gradient descent, no regularization, and no advanced optimizer. The architecture is intentionally simple so every forward and backward step remains inspectable.

Project Completion

Version 5 completes the five-stage Building Neural Networks project: from a single binary neuron, to multiclass classification, one hidden layer, mini-batch training, and finally a small deep neural network with two hidden layers.

Conclusion

Version 5 completes the project by extending the same from-scratch NumPy training pipeline to a deeper 784 -> 128 -> 64 -> 10 network. It preserves the simple, explicit learning logic while adding a second hidden representation and backpropagation through all three trainable layers.